

Uqari PMS — System Documentation

عقاري: Property Management System

Version 3.0 — June 2026

Table of Contents

- [1. System Overview](#)
- [2. Architecture](#)
- [3. Tech Stack](#)
- [4. Project Structure](#)
- [5. Database Schema](#)
- [6. API Reference](#)
- [7. Authentication & Authorization](#)
- [8. Business Logic & Rules](#)
- [9. Financial Engine \(Cron\)](#)
- [10. Frontend Architecture](#)
- [11. Security](#)
- [12. Deployment Guide](#)
- [13. Development Guide](#)
- [14. Testing](#)
- [15. Known Issues & Limitations](#)

1. System Overview

Uqari (عقاري) is a full-stack Property Management System (PMS) designed for elderly landlords in Saudi Arabia. It provides a complete property lifecycle management solution including tenant management, contract handling, automated invoice generation, payment processing (including partial and overpayments with credit tracking), expense tracking with receipt upload, maintenance ticket management, and financial reporting.

Key Design Decisions

Decision	Rationale
better-sqlite3 (synchronous SQLite)	Eliminates callback complexity for single-tenant server; no separate database server needed
Payments in separate table	Full audit trail; supports partial payments without denormalizing invoice data
Invoice generation in contract creation transaction	First invoice becomes visible the instant the contract activates
Soft delete for tenants	Preserves historical contract and invoice references (<code>is_former = 1</code>)
Arabic confirmation word for property deletion	Prevents accidental destruction by elderly users (user must type "حذف")
Automatic <code>updated_at</code> via SQLite triggers	No application code needed; consistent across all tables

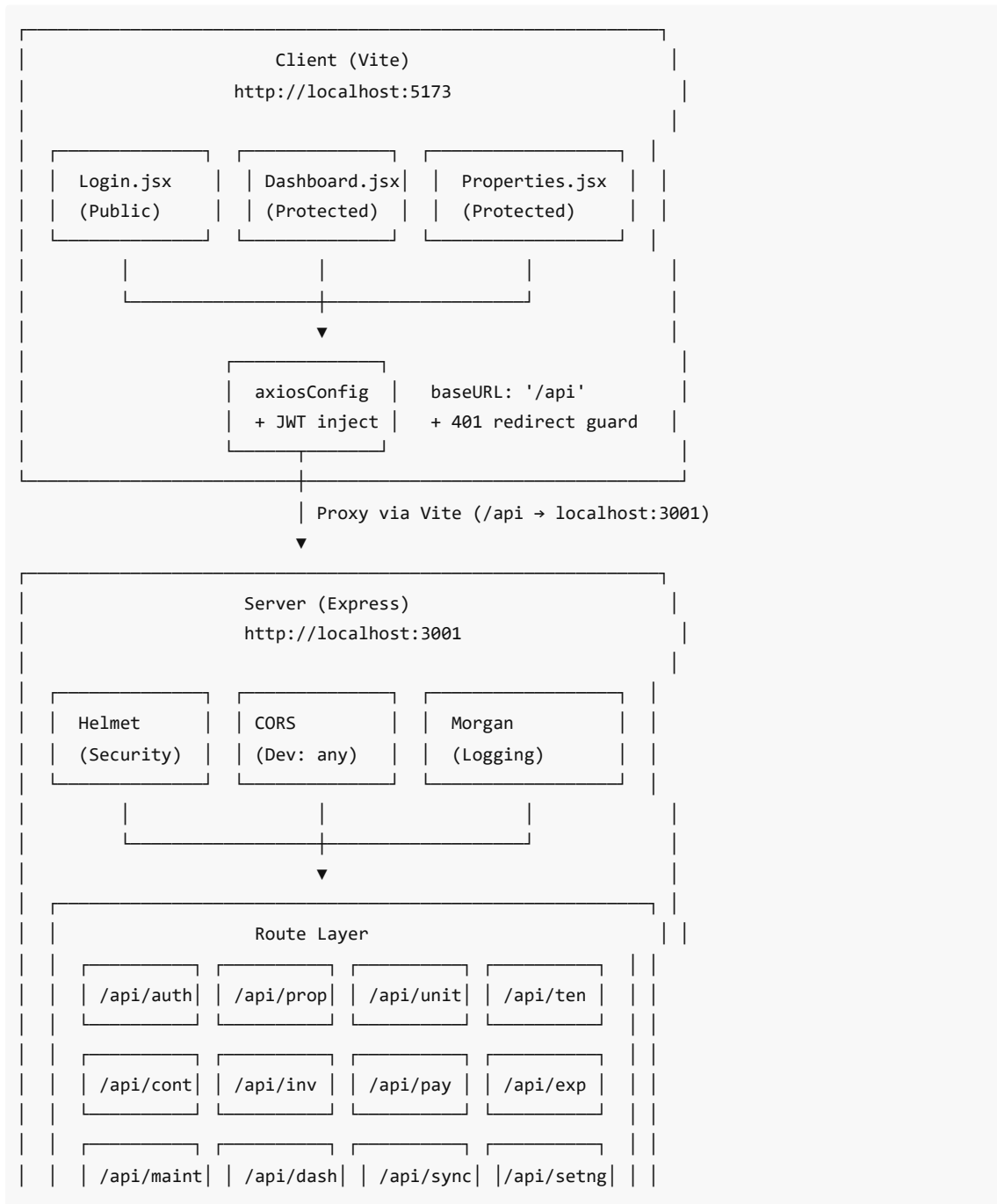
Monorepo with separate package.json files

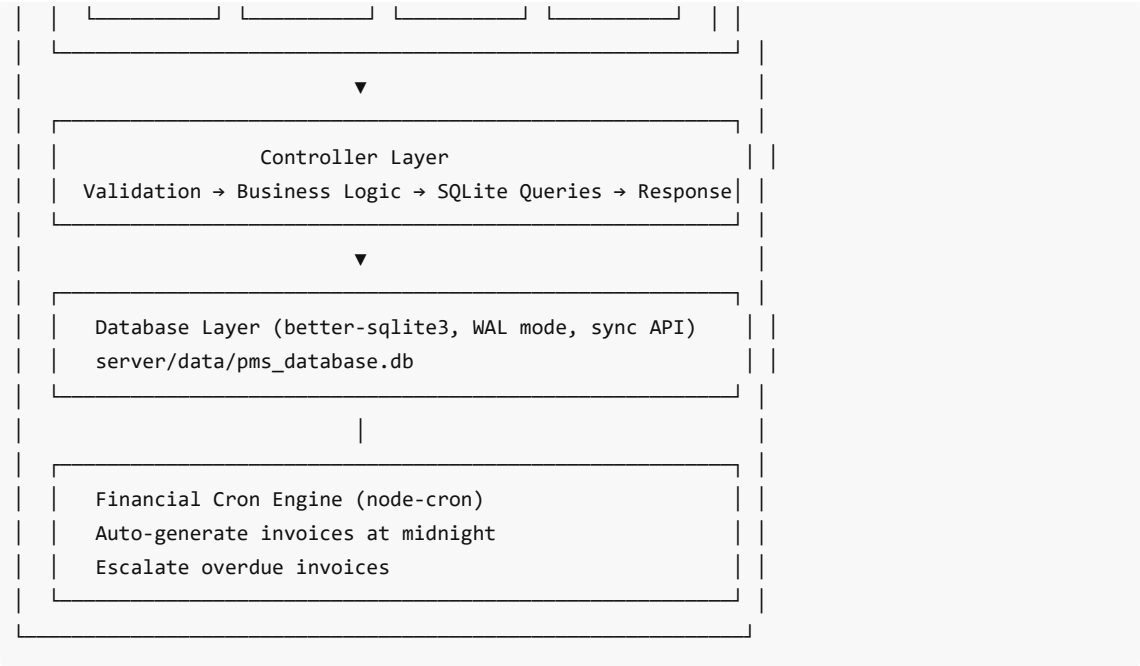
Clean separation without npm workspaces complexity

Target Audience

- Landlords (primary, single-user mode)
- Property managers (future multi-tenant)
- Small to medium property portfolios (1-100 properties)

2. Architecture





Communication Flow

- 1. **Browser** → **Vite Dev Server** (port 5173) — Serves React SPA
- 2. **Axios** → **Vite Proxy** — All `/api/*` requests proxied to Express
- 3. **Express** → **SQLite** — All business logic executed synchronously within routes
- 4. **Cron** → **SQLite** — Runs daily at midnight, checks billing day, generates invoices

3. Tech Stack

Backend

Technology	Version	Purpose
Node.js	18+	Runtime
Express	4.x	Web framework
better-sqlite3	9.x	SQLite3 driver (synchronous)
jsonwebtoken	9.x	JWT generation/verification
bcrypt	5.x	Password/PIN hashing (12 rounds)
multer	1.x	File upload handling (receipts)
node-cron	3.x	Scheduled tasks (invoice generation)
helmet	7.x	HTTP security headers
cors	2.x	Cross-origin resource sharing
morgan	1.x	HTTP request logging

Frontend

Technology	Version	Purpose
React	19.x	UI framework
React Router	7.x	Client-side routing
Axios	1.x	HTTP client
Vite	6.x	Build tool & dev server
@vitejs/plugin-react	4.x	React Fast Refresh

Design

Element	Detail
Fonts	Cairo (Arabic), Inter (numbers)
Theme	Dark navy (#0a1628 background)
Style	Glassmorphism (backdrop-filter blur)
Direction	RTL (right-to-left)
Responsive	Desktop-first, tablet-friendly

4. Project Structure

```
E:\projects\test2\
|
├─ package.json           # Root orchestrator (concurrently)
├─ README.md              # Quick-start instructions
├─ دليل_المستخدم.md      # Arabic user manual
├─ System_Documentation.md # This file
|
├─ server/                # Backend – Express REST API
|   ├─ package.json
|   ├─ server.js           # Entry point, middleware, route mounting
|   ├─ database.js         # SQLite init – 11 tables, 30 indexes, 9 triggers
|   ├─ seed.js             # Standalone seed script
|   |
|   ├─ data/               # SQLite database files
|   |   └─ pms_database.db
|   |
|   ├─ middleware/
|   |   └─ authMiddleware.js # authenticate, optionalAuthenticate, authorize
|   |
|   └─ controllers/        # 12 controllers
|       ├─ authController.js
|       ├─ propertyController.js
|       └─ unitController.js
```

```

| | | tenantController.js
| | | contractController.js
| | | invoiceController.js
| | | paymentController.js
| | | expenseController.js
| | | maintenanceController.js
| | | dashboardController.js
| | | syncController.js
| | | └─ settingsController.js
| |
| | └─ routes/ # 12 route files
| | | └─ authRoutes.js
| | | └─ propertyRoutes.js
| | | └─ unitRoutes.js
| | | └─ tenantRoutes.js
| | | └─ contractRoutes.js
| | | └─ invoiceRoutes.js
| | | └─ paymentRoutes.js
| | | └─ expenseRoutes.js
| | | └─ maintenanceRoutes.js
| | | └─ dashboardRoutes.js
| | | └─ syncRoutes.js
| | | └─ settingsRoutes.js
| |
| | └─ cron/
| | | └─ financialCron.js # Auto-generate + escalate invoices
| |
| | └─ uploads/
| | | └─ receipts/ # Uploaded expense receipts
| |
| └─ client/ # Frontend – React SPA
| | └─ package.json
| | └─ vite.config.js # Vite config + proxy
| |
| | └─ src/
| | | └─ index.jsx # React entry point
| | | └─ index.css # Global styles (Cairo font, RTL)
| | | └─ App.jsx # Router + ProtectedRoute
| | | └─ App.css
| | |
| | | └─ api/
| | | | └─ axiosConfig.js # Axios instance + JWT interceptor
| | |
| | | └─ components/
| | | | └─ layout/
| | | | | └─ Layout.jsx # Sidebar + Outlet
| | | | | └─ Layout.css
| | | |
| | | | └─ shared/
| | | | | └─ Modal.jsx # Reusable modal overlay
| | | | | └─ Modal.css
| | |

```

```
|
|   └─ pages/
|       └─ pages.css          # Shared page styles (tables, forms, buttons, etc.)
|       └─ Login.jsx + Login.css
|       └─ Dashboard.jsx + Dashboard.css
|       └─ Properties.jsx
|       └─ PropertyDetails.jsx
|       └─ UnitForm.jsx
|       └─ Tenants.jsx
|       └─ ContractDetails.jsx
|       └─ Invoices.jsx
|       └─ Expenses.jsx
|       └─ Maintenance.jsx
|       └─ Reports.jsx
|       └─ Settings.jsx
|
| └─ QA_Report.txt           # QA test documentation
|
└─ .gitignore
```

Routing Table (Frontend)

Path	Component	Layout	Access
/login	Login	None	Public
/	Dashboard	Layout	Protected
/properties	Properties	Layout	Protected
/properties/:id	PropertyDetails	Layout	Protected
/units/new	UnitForm	Layout	Protected
/units/:id/edit	UnitForm	Layout	Protected
/tenants	Tenants	Layout	Protected
/contracts/new	ContractDetails	Layout	Protected
/contracts/:id	ContractDetails	Layout	Protected
/invoices	Invoices	Layout	Protected
/expenses	Expenses	Layout	Protected
/maintenance	Maintenance	Layout	Protected
/reports	Reports	Layout	Protected
/settings	Settings	Layout	Protected

Route Mounting (Backend)

Prefix	Router	Middleware
/api/auth	authRoutes	Mixed (public + authenticate)

/api/properties	propertyRoutes	authenticate
/api/units	unitRoutes	authenticate
/api/tenants	tenantRoutes	authenticate
/api/contracts	contractRoutes	authenticate
/api/invoices	invoiceRoutes	authenticate
/api/payments	paymentRoutes	authenticate
/api/expenses	expenseRoutes	authenticate
/api/maintenance	maintenanceRoutes	authenticate
/api/dashboard	dashboardRoutes	authenticate
/api/sync	syncRoutes	authenticate
/api/settings	settingsRoutes	authenticate
/api/health	inline (server.js)	Public
/uploads	express.static	Public

5. Database Schema

Entity-Relationship Diagram (Textual)

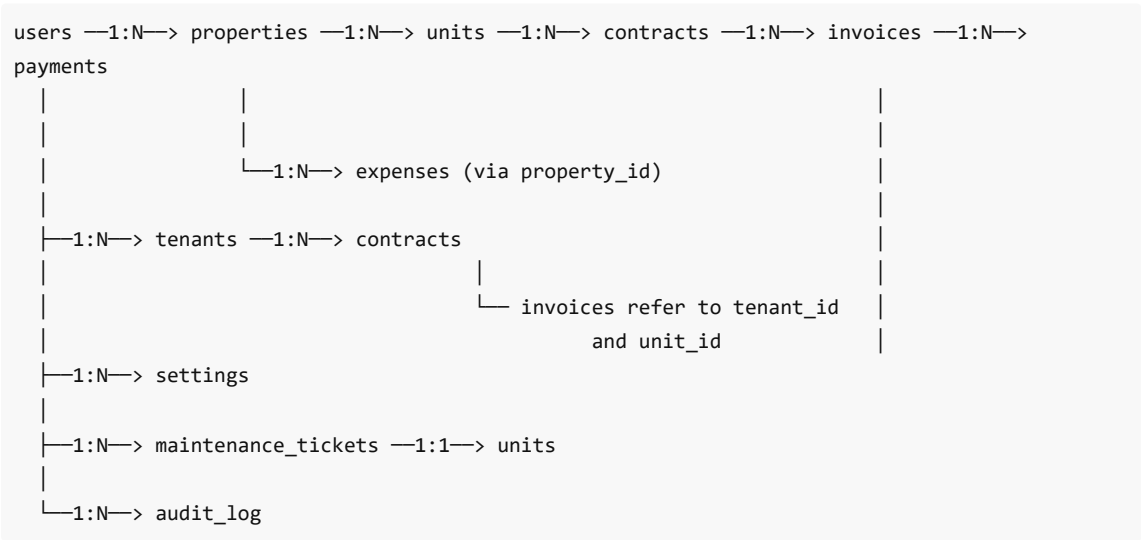


Table Definitions

1. users

Column	Type	Constraints	Description
id	INTEGER	PK AUTOINCREMENT	User ID

name	TEXT	NOT NULL, 2-100 chars	Full name
email	TEXT	NOT NULL, UNIQUE	Login email
password_hash	TEXT	NOT NULL (60 chars)	bcrypt hash
pin_hash	TEXT	DEFAULT NULL	bcrypt hash of 4-digit PIN
role	TEXT	DEFAULT 'landlord', CHECK(IN)	landlord, admin, tenant
is_active	INTEGER	DEFAULT 1, 0/1	Account enabled/disabled
created_at	TEXT	DEFAULT now	ISO 8601
updated_at	TEXT	DEFAULT now (trigger)	Auto-updated

2. properties

Column	Type	Constraints	Description
id	INTEGER	PK AUTOINCREMENT	Property ID
user_id	INTEGER	FK → users(id) ON DELETE CASCADE	Owner
name	TEXT	NOT NULL, 2-100 chars	Property name
city	TEXT	NOT NULL, 2-100 chars	City
notes	TEXT	NULL, max 500 chars	Optional notes
created_at	TEXT	DEFAULT now	ISO 8601
updated_at	TEXT	DEFAULT now (trigger)	Auto-updated

3. units

Column	Type	Constraints	Description
id	INTEGER	PK AUTOINCREMENT	Unit ID
property_id	INTEGER	FK → properties(id) ON DELETE CASCADE	Parent property
unit_number	TEXT	NOT NULL, 1-50 chars	Unit identifier
type	TEXT	DEFAULT 'Apartment'	Apartment, Shop, Room, Villa, Studio
floor	INTEGER	NULL, 0-200	Floor number
monthly_rent	REAL	NOT NULL, 0-999,999,999	Monthly rental amount
status	TEXT	DEFAULT 'Vacant'	Vacant, Occupied, UnderMaintenance
created_at	TEXT	DEFAULT now	ISO 8601

updated_at	TEXT	DEFAULT now (trigger)	Auto-updated
UNIQUE	(property_id, unit_number)		No duplicate unit numbers

4. tenants

Column	Type	Constraints	Description
id	INTEGER	PK AUTOINCREMENT	Tenant ID
user_id	INTEGER	FK → users(id) ON DELETE CASCADE	Landlord
user_account_id	INTEGER	NULL, FK → users(id) ON DELETE SET NULL	Future: self-service account
full_name	TEXT	NOT NULL, 2-100 chars	Full name
phone	TEXT	NOT NULL, 7-20 chars	Primary phone
secondary_phone	TEXT	NULL, 7-20 chars	Alternate phone
national_id	TEXT	NULL, 5-20 chars	Saudi ID / Iqama
notes	TEXT	NULL, max 500 chars	Notes
is_former	INTEGER	DEFAULT 0, 0/1	Soft delete flag
created_at	TEXT	DEFAULT now	ISO 8601
updated_at	TEXT	DEFAULT now (trigger)	Auto-updated

5. contracts

Column	Type	Constraints	Description
id	INTEGER	PK AUTOINCREMENT	Contract ID
tenant_id	INTEGER	FK → tenants(id) ON DELETE CASCADE	Tenant
unit_id	INTEGER	FK → units(id) ON DELETE CASCADE	Unit
start_date	TEXT	NOT NULL (YYYY-MM-DD)	Lease start
end_date	TEXT	NULL (YYYY-MM-DD)	Lease end (null = ongoing)
monthly_rent	REAL	NOT NULL, 0-999,999,999	Agreed rent
status	TEXT	DEFAULT 'Active'	Active, Expired, Terminated
created_at	TEXT	DEFAULT now	ISO 8601
updated_at	TEXT	DEFAULT now (trigger)	Auto-updated

6. invoices

Column	Type	Constraints	Description
--------	------	-------------	-------------

id	INTEGER	PK AUTOINCREMENT	Invoice ID
contract_id	INTEGER	FK → contracts(id) ON DELETE CASCADE	Parent contract
unit_id	INTEGER	NOT NULL	Denormalized for queries
tenant_id	INTEGER	NOT NULL	Denormalized for queries
invoice_number	TEXT	NOT NULL	Format: INV-YYYY-MM-SSS
billing_month	INTEGER	1-12	Month of billing
billing_year	INTEGER	2020-2099	Year of billing
amount	REAL	0-999,999,999	Invoice amount
due_date	TEXT	NOT NULL (YYYY-MM-DD)	Payment deadline
status	TEXT	DEFAULT 'Unpaid'	Unpaid, Paid, Partial, Overdue, Cancelled
notes	TEXT	NULL, max 500 chars	Credit tracking, etc.
created_at	TEXT	DEFAULT now	ISO 8601
updated_at	TEXT	DEFAULT now (trigger)	Auto-updated
UNIQUE	(contract_id, billing_month, billing_year)		One invoice per contract per month

7. payments

Column	Type	Constraints	Description
id	INTEGER	PK AUTOINCREMENT	Payment ID
invoice_id	INTEGER	FK → invoices(id) ON DELETE CASCADE	Target invoice
amount	REAL	>0, ≤999,999,999	Payment amount
payment_date	TEXT	NOT NULL (YYYY-MM-DD)	Date of payment
payment_method	TEXT	NOT NULL	Cash, BankTransfer, Cheque
notes	TEXT	NULL, max 250 chars	Optional notes
created_at	TEXT	DEFAULT now (no trigger)	Created timestamp

8. expenses

Column	Type	Constraints	Description
id	INTEGER	PK AUTOINCREMENT	Expense ID

user_id	INTEGER	FK → users(id) ON DELETE CASCADE	Landlord
property_id	INTEGER	NULL, FK → properties(id) ON DELETE SET NULL	Related property
unit_id	INTEGER	NULL, FK → units(id) ON DELETE SET NULL	Related unit
type	TEXT	NOT NULL	Maintenance, Utilities, Repairs, Cleaning, MunicipalityFees, Insurance, Other
amount	REAL	>0, ≤999,999,999	Expense amount
date	TEXT	NOT NULL (YYYY-MM-DD)	Expense date
description	TEXT	NOT NULL, 3-500 chars	Description
receipt_image_path	TEXT	NULL	Path to uploaded receipt
created_at	TEXT	DEFAULT now	ISO 8601
updated_at	TEXT	DEFAULT now (trigger)	Auto-updated

9. maintenance_tickets

Column	Type	Constraints	Description
id	INTEGER	PK AUTOINCREMENT	Ticket ID
user_id	INTEGER	FK → users(id) ON DELETE CASCADE	Landlord
unit_id	INTEGER	FK → units(id) ON DELETE CASCADE	Affected unit
reported_by	TEXT	NOT NULL, 2-100 chars	Person who reported
description	TEXT	NOT NULL, 10-2000 chars	Problem description
urgency	TEXT	DEFAULT 'Medium'	Low, Medium, High, Emergency
status	TEXT	DEFAULT 'Open'	Open, InProgress, Resolved
issue_image_path	TEXT	NULL	Photo of issue
resolved_at	TEXT	NULL	Resolution timestamp
created_at	TEXT	DEFAULT now	ISO 8601
updated_at	TEXT	DEFAULT now (trigger)	Auto-updated

10. settings (note: table is named settings , key column is key)

Column	Type	Constraints	Description
id	INTEGER	PK AUTOINCREMENT	Setting ID
user_id	INTEGER	FK → users(id) ON DELETE CASCADE	User

key	TEXT	NOT NULL, 1-50 chars	Setting name
value	TEXT	NOT NULL, 0-255 chars	Setting value
created_at	TEXT	DEFAULT now	ISO 8601
updated_at	TEXT	DEFAULT now (trigger)	Auto-updated
UNIQUE	(user_id, key)		One value per key per user

11. audit_log

Column	Type	Constraints	Description
id	INTEGER	PK AUTOINCREMENT	Log ID
user_id	INTEGER	NULL, FK → users(id) ON DELETE SET NULL	User who performed action
action	TEXT	NOT NULL, 2-50 chars	Action name (REGISTER, LOGIN, ADD_PROPERTY, etc.)
target_id	INTEGER	NULL	ID of affected resource
ip_address	TEXT	NULL, max 45 chars	Client IP
user_agent	TEXT	NULL, max 500 chars	Browser/device info
details	TEXT	NULL (JSON)	Action-specific metadata
created_at	TEXT	DEFAULT now	ISO 8601

Indexes

30 composite and single-column indexes across all tables for query optimization including:

- `idx_units_property_status` for filtering units by property + status
- `idx_tenants_name_former` for searching active tenants
- `idx_contracts_active` partial index for active contract lookups
- `idx_invoices_status_due` for overdue invoice queries
- `idx_invoices_user_lookup` for tenant invoice history
- `idx_expenses_user_date` for expense date-range filtering
- `idx_maintenance_tickets_status_urgency` for priority-sorted ticket listing

Triggers

9 `AFTER UPDATE` triggers for automatic `updated_at` timestamp management on: users, properties, units, tenants, contracts, invoices, expenses, maintenance_tickets, settings

6. API Reference

6.1 Authentication

POST `/api/auth/register`

Create a new landlord account.

```
Body:    { "name": "أحمد محمد", "email": "ahmed@example.com", "password": "secret123" }
Success: 201 { success: true, message: "تم إنشاء الحساب بنجاح ✓", data: { token, user: {...} } }
Errors: 409 (duplicate email), 422 (validation)
```

POST /api/auth/login

Authenticate with email and password.

```
Body:    { "email": "ahmed@example.com", "password": "secret123" }
Success: 200 { success: true, message: "تم تسجيل الدخول بنجاح ✓", data: { token, user: {...} } }
Errors: 401 (invalid credentials), 403 (account disabled), 422 (validation)
```

POST /api/auth/verify-pin [Authenticated]

Verify 4-digit PIN for second-factor authentication.

```
Body:    { "pin": "1234" }
Success: 200 { success: true, message: "تم التحقق من الرقم السري بنجاح ✓", data: { verified: true } }
Errors: 400 (no PIN set), 401 (wrong PIN), 422 (format), 429 (locked)
```

POST /api/auth/change-pin [Authenticated]

Change or disable PIN.

```
Body:    { "currentPin": "1234", "newPin": "5678", "confirmPin": "5678" }
          { "currentPin": "1234", "newPin": null, "confirmPin": null } // disable
Success: 200 { success: true, message: "تم تحديث الرقم السري بنجاح ✓" }
Errors: 400, 401, 422
```

6.2 Properties

GET /api/properties [Authenticated]

List all properties with unit count and occupancy.

```
Success: 200 { success: true, data: [{ id, name, city, notes, unitCount, occupiedCount,
createdAt, updatedAt }] }
```

GET /api/properties/:id [Authenticated]

Get property details with all units and current tenant names.

```
Success: 200 { success: true, data: { id, name, city, notes, units: [...], createdAt,
updatedAt } }
```

POST /api/properties [Authenticated]

Create property with auto-generated units.

```
Body:    { "name": "عمارة الخليج", "city": "الرياض", "unitCount": 10, "notes": "..." }
Success: 201 { success: true, message: "تم إضافة العقار بنجاح مع جميع وحداته", data: { id, name, ... } }
Errors: 409 (duplicate name), 422 (validation)
```

PUT /api/properties/:id [Authenticated]

Update property fields.

DELETE /api/properties/:id [Authenticated]

Delete property (requires "حذف" confirmation in body). Checks for active contracts and unpaid invoices.

```
Body:    { "confirm": "حذف" }
Success: 200 { success: true, message: "تم حذف العقار وجميع البيانات المرتبطة به نهائياً" }
Errors:  409 (active contracts exist)
```

6.3 Units

GET /api/units [Authenticated]

List with optional filters: `?property_id=X&status=Vacant|Occupied|UnderMaintenance`

GET /api/units/:id [Authenticated]

Get unit with current contract and last 12 invoices.

POST /api/units [Authenticated]

Create unit.

```
Body:    { "propertyId": 1, "unitNumber": "5 وحدة", "type": "Apartment", "floor": 2,
  "monthlyRent": 2000, "status": "Vacant" }
```

PUT /api/units/:id [Authenticated]

Update unit.

PATCH /api/units/:id/status [Authenticated]

Toggle unit status.

```
Body:    { "status": "Occupied" }
```

DELETE /api/units/:id [Authenticated]

Delete unit. Fails if active contract exists.

6.4 Tenants

GET /api/tenants [Authenticated]

List with filters: `?status=active|former&search=term`

GET /api/tenants/:id [Authenticated]

Get tenant with all contracts + last 20 payments.

POST /api/tenants [Authenticated]

Create tenant. Checks duplicate phone.

```
Body:    { "fullName": "أحمد محمد", "phone": "0555123456", ... }
```

PUT /api/tenants/:id [Authenticated]

Update tenant.

DELETE /api/tenants/:id [Authenticated] (Soft delete)

Set `is_former = 1` . Fails if active contract exists.

6.5 Contracts

POST /api/contracts [Authenticated]

Create contract + auto-generate first invoice inside transaction.

```
Body:    { "tenantId": 1, "unitId": 1, "startDate": "2026-01-01", "monthlyRent": 2000,
  "endDate": "2027-01-01" }
Success: 201 { success: true, message: "...", data: { contract: {...}, invoice: {...} } }
Errors: 409 (tenant has other active contract, unit not vacant)
```

GET /api/contracts [Authenticated]

List with filters: `?status=Active|Expired|Terminated&tenantId=X&unitId=X`

GET /api/contracts/:id [Authenticated]

Get contract with all invoices and paid amounts.

PATCH /api/contracts/:id/terminate [Authenticated]

Terminate contract + vacate unit. Warns about unpaid invoices.

```
Body:    { "terminationDate": "2026-06-01", "reason": "إخلاء" }
Success: 200 { success: true, message: "تم إنهاء العقد وتحرير الوحدة بنجاح." }
```

6.6 Invoices

GET /api/invoices [Authenticated]

List with filters: `?`

`status=Unpaid|Paid|Partial|Overdue|Cancelled&contractId=X&tenantId=X&unitId=X&billingMonth=1-12&billingYear=2020-2099`

GET /api/invoices/overdue [Authenticated]

List overdue invoices (past `due_date`, not fully paid).

GET /api/invoices/:id [Authenticated]

Get invoice with all payments, `totalPaid`, `remaining`.

6.7 Payments

POST /api/payments [Authenticated]

Record payment (full/partial/overpayment with credit tracking).

```
Body:    { "invoiceId": 1, "amount": 2000, "paymentDate": "2026-01-10", "paymentMethod":
  "BankTransfer", "notes": "" }
```

```
Success: 200 { success: true, message: "...", data: { payment: {...}, invoice: {...}, credit?: {...} } }
```

Payment Processing Rules:

Scenario	Amount vs Balance	Invoice Status	Action
Full payment	amount >= balanceDue	Paid	Invoice marked paid
Partial payment	amount < balanceDue	Partial	Partial status
Overpayment	amount > balanceDue	Paid	Credit recorded in invoice.notes
Already paid	—	Paid (existing)	400 error
Cancelled invoice	—	Cancelled (existing)	400 error

GET /api/payments [Authenticated]

List payments with filters: ?invoiceId=X&paymentMethod=Cash|BankTransfer|Cheque&fromDate=YYYY-MM-DD&toDate=YYYY-MM-DD&limit=X

GET /api/payments/credit/:tenantId [Authenticated]

Get tenant credit balance (overpayment tracking).

6.8 Expenses

GET /api/expenses [Authenticated]

List with filters: ?property_id=X&type=...&from_date=...&to_date=...&group_by=month|type|property

GET /api/expenses/summary [Authenticated]

Aggregated summary by property and type.

GET /api/expenses/:id [Authenticated]

Get single expense.

POST /api/expenses [Authenticated + multer]

Create expense with optional receipt upload.

```
Content-Type: multipart/form-data
Fields: propertyId(optional), unitId(optional), type, amount, date, description, receipt(file, optional)
```

PUT /api/expenses/:id [Authenticated]

Update expense.

DELETE /api/expenses/:id [Authenticated]

Delete expense.

6.9 Maintenance Tickets

GET /api/maintenance [Authenticated]

List (sorted by urgency DESC then created_at DESC). Filters: ?
status=Open|InProgress|Resolved&urgency=Low|Medium|High|Emergency&unitId=X

GET /api/maintenance/:id [Authenticated]

Get ticket.

POST /api/maintenance [Authenticated]

Create ticket.

```
Body: { "unitId": 1, "reportedBy": "أحمد", "description": "تسريب مياه من السقف", "urgency":  
"High" }
```

PATCH /api/maintenance/:id [Authenticated]

Update ticket (description, urgency, status).

PATCH /api/maintenance/:id/resolve [Authenticated]

Quick-resolution shortcut (sets status=Resolved, resolved_at=today).

6.10 Dashboard

GET /api/dashboard/summary [Authenticated]

9 KPIs in one call:

```
{  
  "success": true,  
  "data": {  
    "totalProperties": 3,  
    "totalUnits": 15,  
    "occupiedUnits": 10,  
    "occupancyRate": 67,  
    "activeTenants": 10,  
    "totalMonthlyIncome": 40500,  
    "overdueAmount": 5000,  
    "unpaidCount": 3,  
    "pendingTickets": 2,  
    "recentPayments": [...]  
  }  
}
```

GET /api/dashboard/income-expenses [Authenticated]

Monthly income vs expenses. Query: ?months=12

GET /api/dashboard/recent-activity [Authenticated]

Last 5 invoices, maintenance tickets, and expenses.

6.11 Sync

POST /api/sync [Authenticated]

Offline synchronization endpoint. Returns all changes since `lastSyncAt` . Does NOT apply client changes (placeholder implementation).

6.12 Settings

GET /api/settings [Authenticated]

Get all user settings as key-value map.

PUT /api/settings [Authenticated]

Update settings (upserts by key).

```
Body:    { "currency": "SAR", "billingDay": "1", "dueDay": "5", ... }
Errors:  422 (validation of values)
```

6.13 Health

GET /api/health [Public]

Health check. Returns uptime, status, environment.

7. Authentication & Authorization

JWT Token

```
// Payload structure
{
  "userId": 1,
  "role": "landlord",
  "email": "ahmed@example.com",
  "iat": 1749000000,
  "exp": 1749604800
}

// Config
const JWT_SECRET = process.env.JWT_SECRET || 'pms_jwt_secret_change_in_production_2026';
const JWT_EXPIRES_IN = process.env.JWT_EXPIRES_IN || '7d'; // 7 days
```

Middleware Architecture

authenticate(req, res, next) — Used for protected routes:

1. Check for `Authorization: Bearer <token>` header
2. Validate Bearer format
3. Verify JWT signature with `jwt.verify()`
4. Handle specific error types: `TokenExpiredError`, `JsonWebTokenError`, `NotBeforeError`
5. Attach `req.user = { userId, role, email }`
6. Call `next()` ONLY on success — all error paths return `res.status(401).json(...)` with ACID `return`

optionalAuthenticate(req, res, next) — For endpoints that work differently for authenticated users:

- Same flow as authenticate but never blocks
- Sets `req.user = null` on any failure
- Always calls `next()`

authorize(...allowedRoles) — Role-based access:

- Returns middleware that checks `req.user.role`
- Must be used AFTER `authenticate`
- Returns 403 if role not in allowed list

Auth Flow



PIN Lockout Logic

- 3 failed PIN attempts within 5 minutes → 429 "locked for 5 minutes"
- Tracked via `audit_log` with action `FAILED_PIN`
- Lockout check queries count of `FAILED_PIN` entries in last 5 minutes

8. Business Logic & Rules

8.1 Property Deletion Guard

- Requires `{ "confirm": "حذف" }` in request body
- Checks for active contracts on any unit → 409 Conflict
- Checks for unpaid/overdue/partial invoices → 409 Conflict
- CASCADE deletes all related data

8.2 Unit Status Constraints

- Setting `Occupied` requires an active contract on the unit
- Setting `Vacant` requires NO active contract on the unit
- Status toggle cycles: `Vacant` ↔ `Occupied` ↔ `UnderMaintenance`

8.3 Tenant Soft Delete

- `DELETE /api/tenants/:id` sets `is_former = 1`
- Duplicate phone check only applies to active tenants
- Former tenants retain all contract and invoice history

8.4 Contract Creation Transaction

Executed atomically via `db.transaction()` :

1. Update unit: `status = 'Occupied'` , `monthly_rent = parsedMonthlyRent`
2. Insert contract with `status = 'Active'`
3. Insert first month's invoice (status `Unpaid`)
4. If any step fails, all changes are rolled back

8.5 Invoice Number Format

INV-YYYY-MM-SSS

- YYYY: Billing year (e.g., 2026)
- MM: Billing month, zero-padded (e.g., 01)
- SSS: Sequence number, zero-padded to 3 digits (e.g., 001)
- Sequence resets each billing month (COUNT of invoices in that month + 1)

8.6 Payment Processing Business Rules

Within `db.transaction()` :

1. Validate payment amount > 0
2. Calculate existing total paid from payments table
3. Calculate balance due = `invoice.amount - totalPaid`
4. Compare payment amount vs balance:

```
if (amount >= balanceDue) {
  status = 'Paid';
  creditAmount = amount - balanceDue;
  if (creditAmount > 0) {
    // Append credit info to invoice.notes
    invoiceNotes = `زیاده مدفوعه: ${creditAmount} ريال كرسيد دائن...`;
  }
} else {
  status = 'Partial';
}
```

5. Insert payment record
6. Update invoice status
7. Return payment + updated invoice + credit info (if any)

8.7 Due Date Calculation

- Read `dueDay` from `user_settings` (default 5)
- Clamp to month length: `Math.min(dueDay, daysInMonth(year, month))`

9. Financial Engine (Cron)

Schedule

- **Frequency:** Daily at midnight `'0 0 * * *'`
- **Engine:** `node-cron`

- **File:** `server/cron/financialCron.js`

Tasks

1. `autoGenerateInvoices()`

1. Query today's date → get current month and year
2. Read `billingDay` from each user's settings
3. If today's date $\geq$ `billingDay`, generate invoices for all Active contracts:
 - Check no existing invoice for this contract + month/year (UNIQUE constraint)
 - Calculate `due_date` using user's `dueDay` setting (clamped to month length)
 - Generate invoice number: `INV-{year}-{month}-{seq}`
 - Insert invoice with status `Unpaid`
4. Log results in Arabic: `Y فاتورة جديدة لشهر X تم إنشاء`

2. `escalateOverdueInvoices()`

1. Find invoices past `due_date` with status `Unpaid` or `Partial`
2. Update their status to `Overdue`
3. Log overdue count

Error Handling

- All errors caught and logged (do NOT crash the process)
- Each run is isolated — failure in one user's processing doesn't affect others

10. Frontend Architecture

Component Tree

```
<App>
  <BrowserRouter>
    <Routes>
      <Route path="/login" element={<Login />} />
      <Route element={<ProtectedRoute><Layout /></ProtectedRoute>}>
        <Route path="/" element={<Dashboard />} />
        <Route path="/properties/*" element={<Properties | PropertyDetails />} />
        <Route path="/units/*" element={<UnitForm />} />
        <Route path="/tenants" element={<Tenants />} />
        <Route path="/contracts/*" element={<ContractDetails />} />
        <Route path="/invoices" element={<Invoices />} />
        <Route path="/expenses" element={<Expenses />} />
        <Route path="/maintenance" element={<Maintenance />} />
        <Route path="/reports" element={<Reports />} />
        <Route path="/settings" element={<Settings />} />
      </Route>
    </Routes>
  </BrowserRouter>
</App>
```

Axios Configuration

- **baseUrl:** `/api` (relative — works via Vite proxy in dev, Express static in production)
- **Request interceptor:** Injects `Authorization: Bearer <token>` from `localStorage`

- **Response interceptor:** On 401, removes token and redirects to `/login`
- **isRedirecting guard:** Prevents infinite redirect loops

State Management

- No external state management library
- Each page manages its own state via `useState` + `useEffect`
- Data fetching in `useEffect` with cleanup via `abort` pattern or cancelled flag
- Toast notifications managed locally per page

CSS Architecture

- `index.css` — Global: RTL, font imports, scrollbar styling, body defaults
- `Layout.css` — Shell layout: sidebar, main content area
- `Modal.css` — Reusable modal overlay with animations
- `Login.css` — Login page styles
- `Dashboard.css` — Dashboard-specific: metric cards, movements table, skeletons
- `pages.css` — Shared: buttons, tables, forms, badges, filters, cards, modals, toasts

Loading, Error, Empty States

Every page implements 3 states:

1. **Loading:** Skeleton rows or spinner while data is fetched
2. **Error:** Error card with retry button
3. **Empty:** Empty state with CTA button to create first item

11. Security

Implementation

Layer	Measure	Implementation
Transport	Helmet headers	CSP, X-Frame-Options, HSTS, etc.
Authentication	JWT (7d expiry)	RS256-like with symmetric secret
Password storage	bcrypt (12 rounds)	<code>bcrypt.hashSync(password, 12)</code>
PIN storage	bcrypt (12 rounds)	Same as password
PIN lockout	3 attempts / 5 min	Counted via <code>audit_log</code>
Authorization	Role-based	<code>authorize('landlord', 'admin')</code>
Input validation	Server-side	Type checks, length checks, regex patterns
File upload	Multer	5MB limit, allowed types: jpeg/png/gif/pdf
CORS	Explicit origins	Dev: all origins; Prod: whitelist
SQL injection	Parameterized queries	<code>db.prepare().run(...)</code> with <code>?</code> placeholders

Security Headers (Helmet)

```

app.use(helmet({
  contentSecurityPolicy: {
    directives: {
      defaultSrc: ["'self'"],
      scriptSrc: ["'self'"],
      styleSrc: ["'self'", "'unsafe-inline'"],
      imgSrc: ["'self'", "data:", "blob:"],
      connectSrc: ["'self'"],
      // ...
    },
  },
}));

```

What is NOT Implemented

- Rate limiting (express-rate-limit recommended)
- HTTPS (requires reverse proxy)
- SQL injection beyond parameterized queries (no ORM — raw SQL)
- XSS beyond CSP headers (no React server-side rendering)
- CSRF (same-origin via SPA, but no CSRF tokens)

12. Deployment Guide

Development Deployment

```

# 1. Clone and install
cd server && npm install
cd ../client && npm install
cd ..

# 2. Seed database (first time)
cd server && npm run seed

# 3. Start both servers
cd .. && npm run dev

```

Production Deployment

```

# 1. Build frontend
cd client && npm run build
# Creates client/dist/ with optimized static files

# 2. Configure environment
set NODE_ENV=production
set PORT=3000
set JWT_SECRET=<your-strong-secret-here>
set CORS_ORIGIN=https://yourdomain.com

```

```
# 3. Start backend (serves built frontend + API)
cd server && node server.js
```

Environment Variables

Variable	Default	Description
PORT	3001	Server port
NODE_ENV	'development'	'production' enables combined logging
JWT_SECRET	'pms_jwt_secret_change_...'	JWT signing secret
JWT_EXPIRES_IN	'7d'	Token expiry
DATABASE_PATH	./server/data/pms_database.db	SQLite file path
CORS_ORIGIN	localhost origins	Comma-separated allowed origins

Production Considerations

1. **Change JWT_SECRET** to a strong random value
2. **Set CORS_ORIGIN** to your actual domain
3. **Use a reverse proxy** (nginx, Caddy) for:
 - SSL/TLS termination
 - Static file serving (or let Express serve built files)
 - Rate limiting
 - Process management (PM2)
4. **Database backup:** Copy `server/data/pms_database.db` regularly
5. **Log rotation:** Configure Morgan logs to rotate
6. **Upload directory:** Set up `server/uploads/receipts/` with appropriate permissions

Using PM2 (Recommended for Production)

```
npm install -g pm2

# Start with PM2
cd server
pm2 start server.js --name uqari-pms
pm2 save
pm2 startup
```

13. Development Guide

Adding a New API Endpoint

1. Add function in appropriate controller:

```
function myNewFeature(req, res, next) {
  try {
    const db = getDatabase();
```



```

    const userId = req.user.userId;
    // ... business logic ...
    res.status(200).json({ success: true, data: result });
  } catch (error) {
    next(error);
  }
}

```

2. Add route in route file:

```
router.get('/my-feature', authenticate, myNewFeature);
```

3. Mount in server.js:

```
app.use('/api/my-feature', myFeatureRoutes);
```

Adding a New Frontend Page

1. Create component file in `client/src/components/pages/`
2. Import and add route in `App.jsx`
3. Add nav item in `Layout.jsx`

Database Migrations

Since this uses SQLite without an ORM, schema changes are done manually:

```

// In database.js, add new table or ALTER TABLE
db.exec(`ALTER TABLE users ADD COLUMN avatar_url TEXT DEFAULT NULL;`);

```

For production, maintain a migration version table.

NPM Scripts

Script	Location	Description
npm run dev	Root	Start both server + client
npm run dev:server	Root	Start backend only
npm run dev:client	Root	Start frontend only
npm run seed	server	Reset and seed database
npm run build	client	Build for production
npm run lint	—	Not yet configured
npm run test	—	Not yet configured

14. Testing

QA Test Results

22 test cases across 8 modules: **100% pass rate**

Module	Passed	Total
Authentication	5	5
Properties	2	2
Units	1	1
Tenants	1	1
Contracts	2	2
Payments	5	5
Cron	3	3
Dashboard	1	1
UI	2	2

See `QA_Report.txt` for complete test case documentation.

Recommended Test Strategy

- **Unit tests:** Vitest for pure functions (invoice number generation, date calculations)
- **Integration tests:** Supertest for API endpoints
- **End-to-end:** Manual testing via browser with seeded data

15. Known Issues & Limitations

Known Issues

ID	Issue	Impact	Workaround
KNW-001	React StrictMode double-mounts effects in development	Double API calls during dev	Ignore (production single-mount)
KNW-002	Cron skips February billing if billingDay > 28	No invoice for Feb	Set billingDay ≤ 28
KNW-003	Overpayment credit not auto-applied to future invoices	Manual credit tracking	Check invoice notes before billing
KNW-004	Undo button has no backend endpoint	Button shows but does nothing	Manual reversal via DB

Limitations

1. **Single-user mode:** Only one landlord per database instance
2. **No pagination:** List endpoints return all results (fine for <1000 records)
3. **No data export:** No CSV/PDF export for financial reports

4. **No email notifications:** Invoice reminders not sent automatically
5. **No multi-language:** Arabic interface only (no English toggle)
6. **No dark/light mode toggle:** Dark theme only
7. **No backup automation:** Manual database file copy required
8. **Sync endpoint incomplete:** Does not apply client changesets
9. **No soft delete for properties:** Hard CASCADE delete
10. **No invoice cancellation endpoint:** Cannot cancel an invoice after creation

Future Recommendations

1. Implement rate limiting (`express-rate-limit`) on auth endpoints
2. Build Undo payment endpoint (`DELETE /api/payments/:id`)
3. Auto-apply tenant credit when generating new invoices
4. Add CSV/PDF export for financial reports
5. Implement pagination for list endpoints
6. Add search/sort/filter to all tables
7. Implement multi-tenant support via `user_account_id`
8. Add email notification system for invoice reminders
9. Implement audit log viewer in UI
10. Add backup/restore functionality

Appendix A: Controller Function Catalog

authController.js

Function	Method+Path	Auth
register	POST /api/auth/register	Public
login	POST /api/auth/login	Public
verifyPin	POST /api/auth/verify-pin	Authenticated
changePin	POST /api/auth/change-pin	Authenticated

propertyController.js

Function	Method+Path	Auth
listProperties	GET /api/properties	Authenticated
getProperty	GET /api/properties/:id	Authenticated
createProperty	POST /api/properties	Authenticated
updateProperty	PUT /api/properties/:id	Authenticated
deleteProperty	DELETE /api/properties/:id	Authenticated

unitController.js

Function	Method+Path	Auth
listUnits	GET /api/units	Authenticated

getUnit	GET /api/units/:id	Authenticated
createUnit	POST /api/units	Authenticated
updateUnit	PUT /api/units/:id	Authenticated
updateUnitStatus	PATCH /api/units/:id/status	Authenticated
deleteUnit	DELETE /api/units/:id	Authenticated

tenantController.js

Function	Method+Path	Auth
listTenants	GET /api/tenants	Authenticated
getTenant	GET /api/tenants/:id	Authenticated
createTenant	POST /api/tenants	Authenticated
updateTenant	PUT /api/tenants/:id	Authenticated
deleteTenant	DELETE /api/tenants/:id	Authenticated

contractController.js

Function	Method+Path	Auth
createContract	POST /api/contracts	Authenticated
listContracts	GET /api/contracts	Authenticated
getContract	GET /api/contracts/:id	Authenticated
terminateContract	PATCH /api/contracts/:id/terminate	Authenticated

invoiceController.js

Function	Method+Path	Auth
listInvoices	GET /api/invoices	Authenticated
getInvoice	GET /api/invoices/:id	Authenticated
getOverdueInvoices	GET /api/invoices/overdue	Authenticated

paymentController.js

Function	Method+Path	Auth
recordPayment	POST /api/payments	Authenticated
listPayments	GET /api/payments	Authenticated
getTenantCredit	GET /api/payments/credit/:tenantId	Authenticated

expenseController.js

Function	Method+Path	Auth
listExpenses	GET /api/expenses	Authenticated
getExpense	GET /api/expenses/:id	Authenticated
createExpense	POST /api/expenses	Authenticated
updateExpense	PUT /api/expenses/:id	Authenticated
deleteExpense	DELETE /api/expenses/:id	Authenticated
getExpenseSummary	GET /api/expenses/summary	Authenticated

maintenanceController.js

Function	Method+Path	Auth
listTickets	GET /api/maintenance	Authenticated
getTicket	GET /api/maintenance/:id	Authenticated
createTicket	POST /api/maintenance	Authenticated
updateTicket	PATCH /api/maintenance/:id	Authenticated
resolveTicket	PATCH /api/maintenance/:id/resolve	Authenticated

dashboardController.js

Function	Method+Path	Auth
getSummary	GET /api/dashboard/summary	Authenticated
getIncomeExpenses	GET /api/dashboard/income-expenses	Authenticated
getRecentActivity	GET /api/dashboard/recent-activity	Authenticated

syncController.js

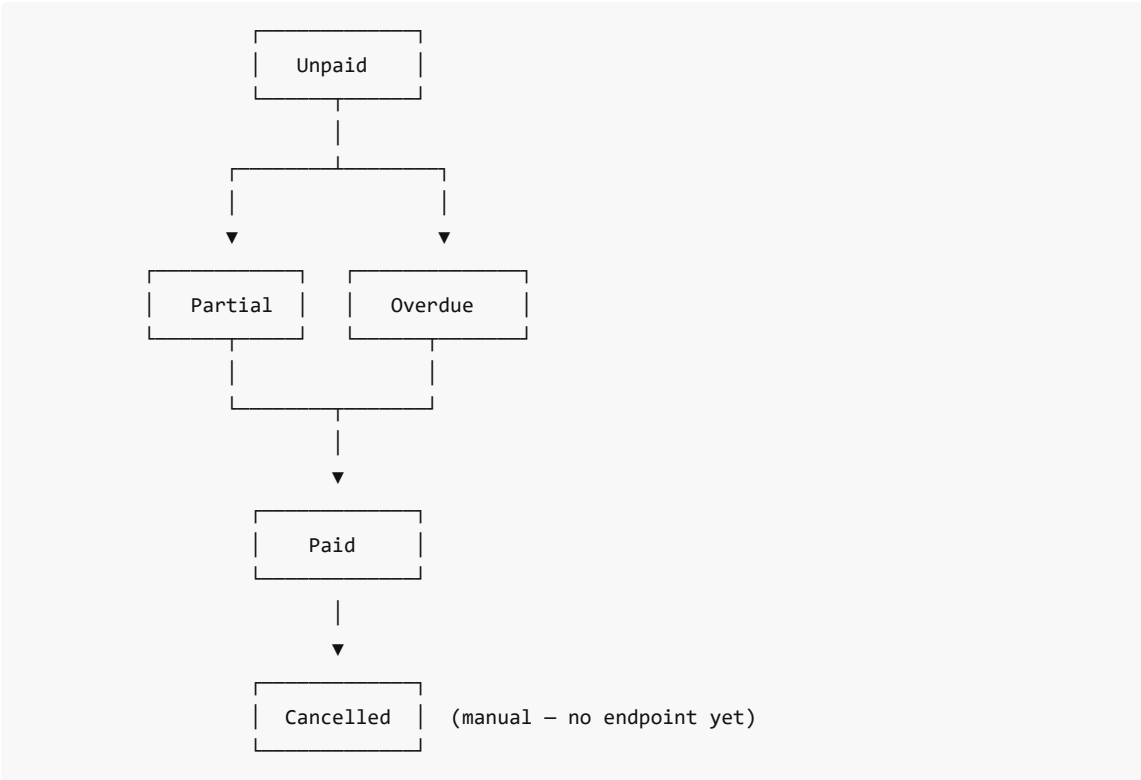
Function	Method+Path	Auth
syncData	POST /api/sync	Authenticated

settingsController.js

Function	Method+Path	Auth
getSettings	GET /api/settings	Authenticated
updateSettings	PUT /api/settings	Authenticated

Total: 47 endpoint handlers across 12 controllers

Appendix B: Invoice Status Lifecycle



Note: **Overdue** status is set automatically by the cron engine (not by payment processing). The payment system only sets **Unpaid** , **Partial** , or **Paid** .

Appendix C: Expense Type Labels

DB Value	Arabic Label	Category
Maintenance	صيانة	Repairs & upkeep
Utilities	خدمات	Electricity, water, gas
Repairs	إصلاحات	Fixes and renovations
Cleaning	تنظيف	Cleaning services
MunicipalityFees	رسوم بلدية	Municipal charges
Insurance	تأمين	Property insurance
Other	أخرى	Miscellaneous

Appendix D: Maintenance Urgency Definitions

Level	Response Time	Color	Use Case
Emergency	Immediate (hours)	Red	Water leak, electrical hazard, broken lock

High	Within 24h	Orange	AC failure, plumbing issue, no hot water
Medium	Within 3 days	Yellow	Paint touch-up, fixture replacement
Low	Within 1 week	Gray	Cosmetic upgrades, non-urgent requests

Document generated June 2026. For project issues, visit <https://github.com/anomalyco/opencode/issues>